

Regresja - Notatki z wykładu (tłumaczenie PL)

Materiał źródłowy: Dr. Brian Mc Ginley (Week 1 – Regression.pdf) · Tłumaczenie: Marcin
27 września 2026

Slajdy są w kolejności wykładu. Elementy graficzne, których nie da się przepisać (wykresy, zdjęcia), są opisane w nawiasach kwadratowych. Wzory matematyczne i fragmenty kodu pozostawiono w oryginalnej notacji.

1. Regresja (strona tytułowa)

- Dr. Brian Mc Ginley

2. Zmienne niezależne / zależne

- Krótka notatka o zmiennych niezależnych i zależnych
- Przykład: wartość domu w czasie
 - Mamy dwie zmienne:
 - Wartość domu (V)
 - Czas (t)
 - Ponieważ obie się zmieniają, są zmiennymi
 - Wartość jest zmienną **zależną**, ponieważ wartość domu **zależy od** czasu, w którym go kupiono
 - Czas jest **niezależny**.
 - Na wykresach oś pozioma zwykle reprezentuje zmienną niezależną, a oś pionowa – zmienną zależną

3. Związki liniowe (1)

- W tym module będziemy głównie zajmować się związkami liniowymi, aby dokonywać predykcji.
- **Definicja:**
 - Związek liniowy: zmiana wartości x zawsze powoduje taką samą, proporcjonalną zmianę wartości y .
- Rozważmy poniższą tabelę:
 - Jaka jest wartość y , gdy $x = 3$?
 - Jaka wartość x da $y = 44$?

x	1	2	3	4	5	6	...
y	5	8	?	14	17	20	... 44

4. Związki liniowe (2)

- Gdy jesteśmy przekonani, że jakiś związek istnieje, możemy ustalić jego dokładny charakter i użyć go do przewidywania wartości jednej zmiennej odpowiadających dowolnym wartościom drugiej.
- Dokładność związku widać na diagramie. Naniesione punkty leżą wzdłuż

wyimaginowanej prostej. Gdybyśmy poprowadzili przez nie prostą, moglibyśmy jej użyć do **dokładnych predykcji nawet bez użycia wzoru**.

[Wykres punktowy: punkty A-H leżą niemal idealnie na rosnącej prostej; oś pozioma 0-14, oś pionowa 0-ok. 44.]

5. Związki liniowe (3)

- Czy potrafisz wymyślić wzór dla poprzedniego wykresu?

$$y = mx + c$$
$$y = 3x + 2$$

dla tej konkretnej prostej. Możemy teraz użyć tego wzoru do „przewidywania” wartości.

- To bardzo wcześnie nauczane równanie prostej jest podstawą regresji i wielu analiz uczenia maszynowego.

6. Regresja

- Tutaj staramy się precyzyjnie określić wartość ciągłą (lub wartości), zamiast przewidywać klasę lub kategorię.
- **Przykład: jak upiec indyka**
 - Istnieje reguła kciuka przy pieczeniu indyka.
 - Włóż go do nagrzanego piekarnika na 20 minut na funt plus 20 minut.
 - Skąd wzięła się ta zasada?
 - Czy jest poprawna?
 - Co mogłoby się stać, gdyby była błędna?

7. Przykład: jak upiec indyka (1)

- To model liniowy, prawdopodobnie oparty na analizie regresji liniowej dla indyków o różnych wagach. Model matematyczny to

$$t = mw + c$$

- gdzie t to czas pieczenia, w to waga w funtach, m to nachylenie prostej, a c to punkt przecięcia z osią y .
- Pief Panofsky się z tym nie zgodził i użył następującego modelu:

$$t = w^{(2/3)} / 1,5$$

- Przeanalizujmy to.

8. Przykład: jak upiec indyka (2)

- Po pierwsze zauważ, że w modelu liniowym trzeba by piec „wyimaginowanego” indyka przez 20 minut (przy wadze zero).
- Między 1 a 6 funtami oba modele są bardzo zbliżone. Przy 16 funtach różnica między nimi wynosi już 90 minut.
 - Który jest bezpieczniejszy?

- Który smakuje lepiej?
- Używaj termometru do mięsa.

[Wykres „Cooking times for Turkey”: oś pozioma – waga indyka w funtach (0–18), oś pionowa – minuty (0–400); dwie linie: model liniowy (niebieska) i model Piefa Panofsky’ego (pomarańczowa), rozjeżdżające się przy wyższych wagach.]

9. Dopasowanie modelu liniowego

- Powyższy przykład pokazuje wynik modelu liniowego, ale nie to, jak do niego doszliśmy.
- Aby to zrobić, musimy zebrać dane i dopasować do nich najlepszą prostą.
- W powyższym przypadku oznaczałoby to upieczenie wielu indyków o różnych wagach i ugotowanie ich wszystkich przy użyciu jakiejś skalibrowanej metody, np. termometru do mięsa (lub doświadczonego kucharza-eksperta).
- Zbieramy „poprawną” odpowiedź i wykorzystujemy ją do zbudowania modelu, czyli do wyznaczenia parametrów.
- Spójrzmy na trochę danych.

10. Przykład: jak upiec indyka (3)

- Mając dane po lewej, gdybyś miał dziewięciofuntowego indyka, jak długo powinienś go piec?
- Czy to zadanie dla uczenia nadzorowanego, czy nienadzorowanego?
- Jeśli założymy model liniowy, jakie parametry musimy się nauczyć?

$$t = mw + c$$

[Ten sam wykres „Cooking times for Turkey” co powyżej.]

11. Przykład: jak upiec indyka (4)

Waga w funtach	1,1	1,3	1,7	2	2,1	2,3	...
Czas pieczenia	39,4	48,8	55,8	63,0	69,5	67,7	...

- Zatem w to nasza zmienna wejściowa/cecha.
- t to nasze wyjście i można je uznać za cel regresji (regression target), którego użyjemy do trenowania modelu.
- Musimy nauczyć się m i c .
- Będziemy też używać oznaczenia M na liczbę próbek treningowych.
 - (w, t) – jeden przykład treningowy.
 - $(w^{(i)}, t^{(i)})$ – i -ty przykład treningowy.

12. Notacja

- Zauważysz, że na poprzednich slajdach używaliśmy nazw zmiennych, które albo pasują do problemu, albo są powszechnie stosowane w matematyce.
- Nie zawsze jednak będziemy mieć do czynienia z modelami liniowymi, więc będziemy używać notacji, którą da się skalować do większych modeli.
- Nie ma jednej ustalonej notacji, więc tam, gdzie to możliwe, będziemy stosować taką, jaka jest używana w bibliotece scikit-learn.
- Dlatego będziemy używać w do oznaczenia parametrów/wag, które będziemy

trenować. Najczęściej y będzie naszym wyjściem, a x – wejściem.

- Zatem nasze powyższe równanie przyjmie teraz postać:

$$y = w_0 + w_1 x$$

13. Ostrzeżenie przed użyciem regresji liniowej

- Regresja liniowa opiera się na założeniu, że punkty danych są rozproszone wokół prostej. Często jednak punkty danych są rozproszone wokół krzywej, np.:

[Zdjęcie z podręcznika: dwa wykresy punktowe w kształcie łuku – po lewej same punkty, po prawej te same punkty z dopasowaną, rosnącą prostą.]

- Nadal można obliczyć współczynniki (w_0 i w_1), które dadzą powyższą prostą. Widać jednak, że to niewłaściwe dopasowanie – prosta sugeruje, że wartości y stale rosną, podczas gdy krzywa pokazuje, że po pewnym punkcie zaczną faktycznie maleć.
- **Kryterium wyboru prostej regresji**
 - Zanim wyznaczysz prostą regresji dla zbioru punktów danych, narysuj wykres rozrzutu. Jeśli punkty danych nie wyglądają na rozproszone wokół prostej, nie wyznaczaj prostej regresji.

14. Inne rodzaje regresji

- To był przykład prostej regresji liniowej (*Simple Linear Regression*). Jest tylko jedna cecha niezależna, ale oczywiście świat rzeczywisty tak nie działa.
- Może istnieć regresja wielomianowa:

$$y = w_0 + w_1 x + w_2 x^2 + \dots + w_n x^n$$

- Regresja wieloraka (*Multiple Linear Regression*), gdzie zamiast pojedynczej cechy x mamy wektor cech $\mathbf{x} = (x_1, x_2, x_3, x_4, \dots)^T$:

$$y = w_0 + w_1 x_1 + w_2 x_2 + w_3 x_3 + \dots$$

- Przyjrzymy się im szczegółowo później, ale pamiętaj, że wagi (w) w obu przypadkach są nadal liniowe. Wagi możemy też zapisać jako wektor:

$$\mathbf{w} = (w_0, w_1, w_2, w_3, w_4, \dots)^T$$

15. Błędy

- Pierwszy pokazany przeze mnie przykład miał wszystkie punkty danych (dane treningowe) leżące dokładnie na prostej. W rzeczywistości tak to nie działa – predykcja nie będzie dokładnie taka sama jak to, co podaliśmy do modelu podczas treningu. Pomyśl o przykładzie, który pokazywałem: liczba znajomych a dzienne minuty online.
- **Przykład:**

[Dwa wykresy punktowe z dopasowaną prostą: lewy $y = 3x$, prawy $y = 2x - 3$ – punkty rozproszone wokół prostej, nieleżące dokładnie na niej.]

- A więc istnieje pewna wielkość błędu modelu względem rzeczywistych danych.

16. Prosta regresja liniowa

- Funkcja dokonuje predykcji dla każdego zaobserwowanego punktu danych.
- Obserwację oznaczamy przez y , a predykcję – przez $\hat{y}$.
- Prosta z najmniejszym błędem – to właśnie ten model chcemy uzyskać!

[Diagram: punkty rozrzucone wokół prostej regresji; powiększony fragment pokazuje jeden punkt jako „Observation: y ” (obserwacja) i odpowiadający mu punkt na prostej jako „Prediction: $\hat{y}$ ” (predykcja), z poziomem odniesienia oznaczonym jako „Zero”.]

17. Trenowanie modelu - minimalizacja błędu

- Trenowanie modelu oznacza znalezienie najlepszej prostej, która minimalizuje błąd dla naszej próbki treningowej. Zależy to od funkcji straty/błędu (*Loss/Error function*).
- Mierzmy różnicę między tym, co model przewiduje, a tym, jaka była rzeczywista wartość w próbce treningowej.
- Istnieje wiele funkcji straty do wyboru, a wybór niewłaściwej może mieć równie duże znaczenie, jak wybór niewłaściwego typu modelu.
- Typowym przykładem jest błąd średniokwadratowy (*mean squared error*):

$$L = (1/2m) \cdot \sum (\hat{y}_i - y_i)^2 \quad (\text{suma po } i = 0 \dots m)$$

- gdzie m to liczba próbek, $\hat{y}_i$ to predykcja dla i -tej próbki, a y_i to poprawna odpowiedź dla tej próbki.
- Uwaga: czynnik $1/2m$ został wybrany wyłącznie dla wygody, żeby zapis matematyczny wyglądał „ładniej”. Minimalizacja L bez tego ułamka da nam te same wartości w .

18. Trenowanie modelu

- Chcemy więc znaleźć te konkretne parametry (powiedzmy w_0 i w_1 w najprostszym z widzianych przypadków), które dają nam najmniejszą wartość L .
- Dla dotychczasowych przykładów regresji liniowej jest to dość proste – istnieje znany wzór, albo możemy posłużyć się algebrą.
- Chcemy jednak, żeby ta idea działała także dla dużo bardziej skomplikowanych modeli, z większą liczbą parametrów.
- Idea ta rzeczywiście działa w kolejnych algorytmach ML: chodzi o znalezienie parametrów $w_0, w_1, w_2, \dots$, które dają najmniejsze L – zmienia się jedynie technika znajdowania tych parametrów, która w bardziej skomplikowanych modelach może zająć nieco więcej czasu. Dotyczy to nawet głębokiego uczenia (sieci neuronowych).
- Zwykle stosuje się do tego metodę zwaną **spadkiem gradientu** (*gradient descent*) (o tym później!), która wykonuje wiele iteracji, aby znaleźć najlepsze parametry.
 - Kiedy używamy metody `.fit()` w scikit-learn i TensorFlow, to właśnie wtedy to się dzieje.

19. Budowanie modelu

- Mamy zestaw danych, czyli parę informacji. Znamy poprawną odpowiedź dla tego zestawu.
 - X to zbiór wszystkich wejść (macierz).

- y to zbiór wszystkich odpowiadających im zmiennych odpowiedzi.
- Konkretny y_i to wynik dla konkretnego x_i .

```
from sklearn import linear_model

lr = linear_model.LinearRegression()
lr.fit(X, y)
pred = lr.predict(X)
```